

MYSQL ASSIGNMENT 2 - **CLAUSES AND JOINS**

BY

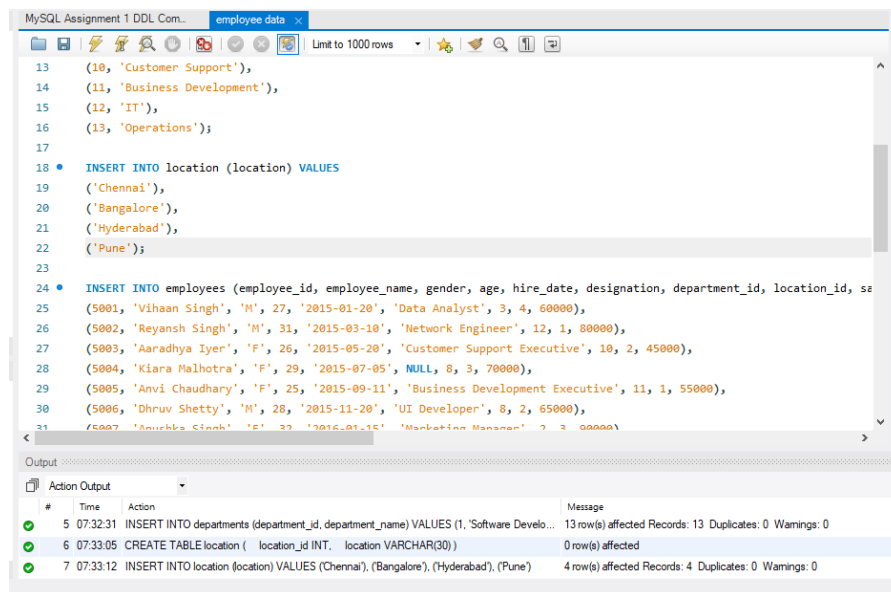
ATHIRA SASEENDRAN

INTRODUCTION

MySQL is a widely used relational database management system that allows users to store, manage, and retrieve data efficiently. This assignment focuses on performing various SQL operations using MySQL Workbench on an employee database. The tasks include retrieving distinct values, filtering and sorting data, using aggregate functions, grouping records, applying conditions with HAVING, and working with different types of joins such as INNER JOIN, LEFT JOIN, and RIGHT JOIN. Through these exercises, the assignment provides practical understanding of how SQL queries can be used to manage and analyse relational data.

Populating the database using the provided **Employee Data** DML statements

This assignment was continued from the previous assignment, where the database, tables, and required constraints were created. The provided **Employee Data** DML insert statements were then used to populate the tables with the required department, location, and employee records. After inserting the data, the SQL queries given in this assignment were executed on the populated database.



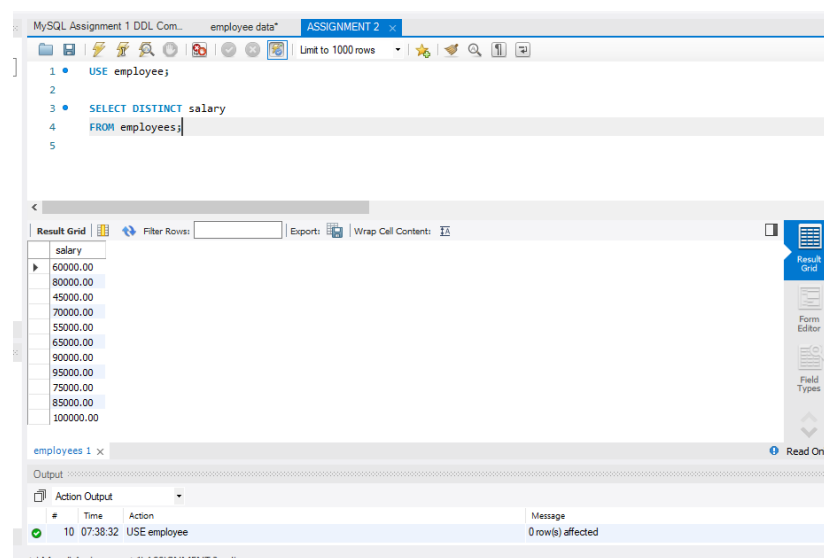
```
13 (10, 'Customer Support'),
14 (11, 'Business Development'),
15 (12, 'IT'),
16 (13, 'Operations');
17
18 • INSERT INTO location (location) VALUES
19 ('Chennai'),
20 ('Bangalore'),
21 ('Hyderabad'),
22 ('Pune');
23
24 • INSERT INTO employees (employee_id, employee_name, gender, age, hire_date, designation, department_id, location_id, salary) VALUES
25 (5001, 'Vihaan Singh', 'M', 27, '2015-01-20', 'Data Analyst', 3, 4, 60000),
26 (5002, 'Reyansh Singh', 'M', 31, '2015-03-10', 'Network Engineer', 12, 1, 80000),
27 (5003, 'Aaradhya Iyer', 'F', 26, '2015-05-20', 'Customer Support Executive', 10, 2, 45000),
28 (5004, 'Kiara Malhotra', 'F', 29, '2015-07-05', NULL, 8, 3, 70000),
29 (5005, 'Anvi Chaudhary', 'F', 25, '2015-09-11', 'Business Development Executive', 11, 1, 55000),
30 (5006, 'Dhruv Shetty', 'M', 28, '2015-11-20', 'UI Developer', 8, 2, 65000),
31 (5007, 'Aurubha Singh', 'F', 32, '2015-01-15', 'Marketing Manager', 2, 3, 90000);
```

Output

#	Time	Action	Message
5	07:32:31	INSERT INTO departments (department_id, department_name) VALUES (1, 'Software Development'), (2, 'Marketing'), (3, 'Human Resources'), (4, 'Finance'), (5, 'Operations'), (6, 'Customer Support'), (7, 'Business Development'), (8, 'IT'), (9, 'Sales'), (10, 'Customer Support'), (11, 'Business Development'), (12, 'IT'), (13, 'Operations');	13 row(s) affected Records: 13 Duplicates: 0 Warnings: 0
6	07:33:05	CREATE TABLE location (location_id INT, location VARCHAR(30));	0 row(s) affected
7	07:33:12	INSERT INTO location (location) VALUES (Chennai), (Bangalore), (Hyderabad), (Pune);	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0

1. DISTINCT VALUES

To retrieve the distinct salary values from the employees table.



```
1 • USE employee;
2
3 • SELECT DISTINCT salary
4 FROM employees;
5
```

Result Grid

salary
60000.00
80000.00
45000.00
70000.00
55000.00
65000.00
90000.00
95000.00
75000.00
85000.00
100000.00

employees 1 x

Output

#	Time	Action	Message
10	07:38:32	USE employee	0 row(s) affected

2. ALIAS (AS)

To display the age and salary columns using the specified aliases.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
3 • SELECT DISTINCT salary
4 FROM employees;
5
6 • SELECT age AS Employee_Age,
7 salary AS Employee_Salary
8 FROM employees;
```

The result grid displays the output of the second query, showing two columns: Employee_Age and Employee_Salary. The data is as follows:

Employee_Age	Employee_Salary
27	60000.00
31	80000.00
26	45000.00
29	70000.00
25	55000.00
28	65000.00
32	90000.00
27	70000.00
30	55000.00
29	80000.00
28	60000.00
26	95000.00
33	75000.00
30	60000.00

The output pane shows the execution of the query: "SELECT DISTINCT salary FROM employees LIMIT 0, 1000" returned 11 row(s).

3. WHERE CLAUSE & OPERATORS

- a) To retrieve employees whose salary is greater than ₹50,000 and who were hired before 1 January 2016.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
6 • SELECT age AS Employee_Age,
7 salary AS Employee_Salary
8 FROM employees;
9
10 • SELECT *
11 FROM employees
12 WHERE salary > 50000
13 AND hire_date < '2016-01-01';
```

The result grid displays the output of the third query, showing columns: employee_id, employee_name, gender, age, hire_date, designation, department_id, location_id, and salary. The data is as follows:

employee_id	employee_name	gender	age	hire_date	designation	department_id	location_id	salary
5001	Vihaan Singh	M	27	2015-01-20	Data Analyst	3	4	60000.00
5002	Reyansh Singh	M	31	2015-03-10	Network Engineer	12	1	80000.00
5004	Klara Malhotra	F	29	2015-07-05	Network Engineer	8	3	70000.00
5005	Anvi Chaudhary	F	25	2015-09-11	Business Development Executive	11	1	55000.00
5006	Dhruv Shetty	M	28	2015-11-20	UI Developer	8	2	65000.00

The output pane shows the execution of the query: "SELECT age AS Employee_Age, salary AS Employee_Salary FROM employees LIMIT 0, 1000" returned 30 row(s).

- b) To identify the employee whose designation is missing. IS NULL is used to find records where the designation value is missing.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following queries:

```
10 • SELECT *
11 FROM employees
12 WHERE salary > 50000
13 AND hire_date < '2016-01-01';
14
15 • SELECT *
16 FROM employees
17 WHERE designation IS NULL;
```

The Result Grid shows one row of data:

employee_id	employee_name	gender	age	hire_date	designation	department_id	location_id	salary
5004	Klara Malhotra	F	29	2015-07-05		8	3	70000.00

The Output pane shows the execution of the second query:

#	Time	Action	Message
13	07:45:33	SELECT * FROM employees WHERE salary > 50000 AND hire_date < '2016-01-01' LIMIT 0...	5 row(s) returned

The UPDATE statement changes the missing designation of employee 5004 to **Data Scientist**.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following queries:

```
12 WHERE salary > 50000
13 AND hire_date < '2016-01-01';
14
15 • SELECT *
16 FROM employees
17 WHERE designation IS NULL;
18
19 • UPDATE employees
20 SET designation = 'Data Scientist'
21 WHERE employee_id = 5004
22 LIMIT 1;
```

The Result Grid is empty.

The Output pane shows the execution of the UPDATE statement:

#	Time	Action	Message
20	07:53:17	UPDATE employees SET designation = 'Data Scientist' WHERE employee_id = 5004 LIMIT 1	0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0
21	07:54:05	SELECT * FROM employees WHERE designation IS NULL LIMIT 0, 1000	0 row(s) returned

SORTING AND GROUPING DATA

I. ORDER BY

To sort employees by department ID and salary...

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
26 WHERE employee_id = 5004;  
27  
28 • SELECT *  
29 FROM employees  
30 ORDER BY department_id ASC, salary DESC;
```

The 'Result Grid' tab is active, displaying a table with 30 rows of employee data, sorted by department_id in ascending order and salary in descending order within each department. The columns are: employee_id, employee_name, gender, age, hire_date, designation, department_id, location_id, and salary.

The 'Output' tab shows the execution log:

#	Time	Action	Message
22	07:56:16	SELECT * FROM employees WHERE employee_id = 5004 LIMIT 0, 1000	1 row(s) returned
23	07:57:39	SELECT * FROM employees ORDER BY department_id ASC, salary DESC LIMIT 0, 1000	30 row(s) returned

II. LIMIT

Displaying the first five employees hired in 2018

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
29 FROM employees  
30 ORDER BY department_id ASC, salary DESC;  
31  
32 • SELECT *  
33 FROM employees  
34 WHERE hire_date >= '2018-01-01'  
35 AND hire_date < '2019-01-01'  
36 ORDER BY hire_date ASC  
37 LIMIT 5;
```

The 'Result Grid' tab is active, displaying a table with 5 rows of employee data, sorted by hire_date in ascending order. The columns are: employee_id, employee_name, gender, age, hire_date, designation, department_id, location_id, and salary.

The 'Output' tab shows the execution log:

#	Time	Action	Message
23	07:57:39	SELECT * FROM employees ORDER BY department_id ASC, salary DESC LIMIT 0, 1000	30 row(s) returned
24	08:01:02	SELECT * FROM employees WHERE hire_date >= '2018-01-01' AND hire_date < '2019-01-01' LIMIT 0, 1000	5 row(s) returned

III. AGGREGATE FUNCTIONS

a). Calculating the total salary of the Finance department. Using SUM()

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
33 FROM employees
34 WHERE hire_date >= '2018-01-01'
35 AND hire_date < '2019-01-01'
36 ORDER BY hire_date ASC
37 LIMIT 5;
38
39 • SELECT SUM(salary) AS Total_Finance_Salary
40 FROM employees
41 WHERE department_id = 7;
```

The Result Grid shows the output of the query:

Total_Finance_Salary
170000.00

The Output pane shows the execution log:

#	Time	Action	Message
2	08:05:21	USE employee	0 row(s) affected
3	08:05:24	SELECT SUM(salary) AS Total_Finance_Salary FROM employees WHERE department_id = ...	1 row(s) returned

b). Finding the minimum age among employees. Using MIN().

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following queries:

```
39 • SELECT SUM(salary) AS Total_Finance_Salary
40 FROM employees
41 WHERE department_id = 7;
42
43 • SELECT MIN(age) AS Minimum_Age
44 FROM employees;
45
46 • SELECT MIN(age) AS Minimum_Age
47 FROM employees;
```

The Result Grid shows the output of the first query:

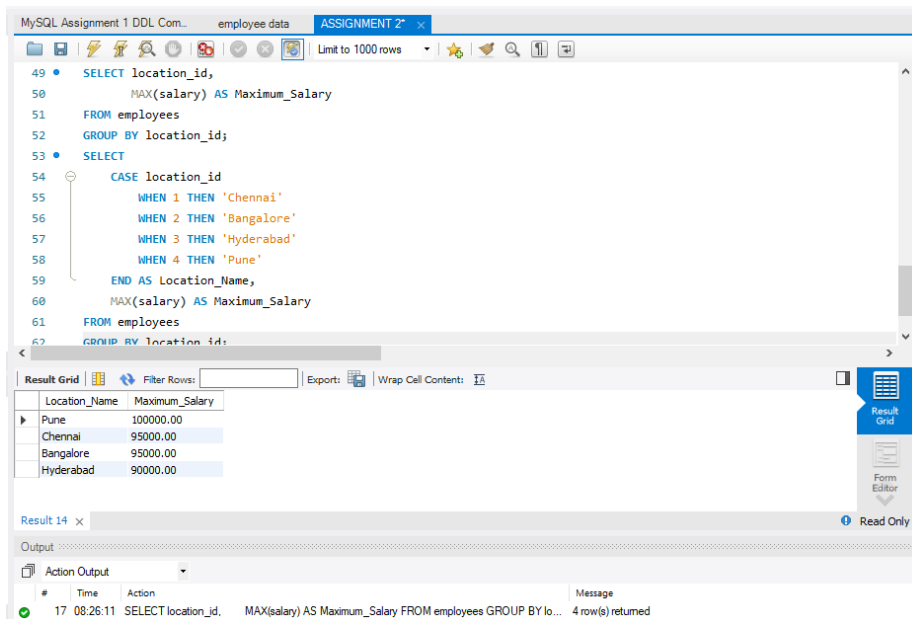
Minimum_Age
25

The Output pane shows the execution log:

#	Time	Action	Message
3	08:05:24	SELECT SUM(salary) AS Total_Finance_Salary FROM employees WHERE department_id = ...	1 row(s) returned
4	08:07:33	SELECT MIN(age) AS Minimum_Age FROM employees LIMIT 0, 1000	1 row(s) returned

IV. GROUP BY

a). Finding the maximum salary for each location



```
49 • SELECT location_id,
50         MAX(salary) AS Maximum_Salary
51     FROM employees
52     GROUP BY location_id;
53 • SELECT
54     CASE location_id
55     WHEN 1 THEN 'Chennai'
56     WHEN 2 THEN 'Bangalore'
57     WHEN 3 THEN 'Hyderabad'
58     WHEN 4 THEN 'Pune'
59     END AS Location_Name,
60     MAX(salary) AS Maximum_Salary
61     FROM employees
62     GROUP BY location_id;
```

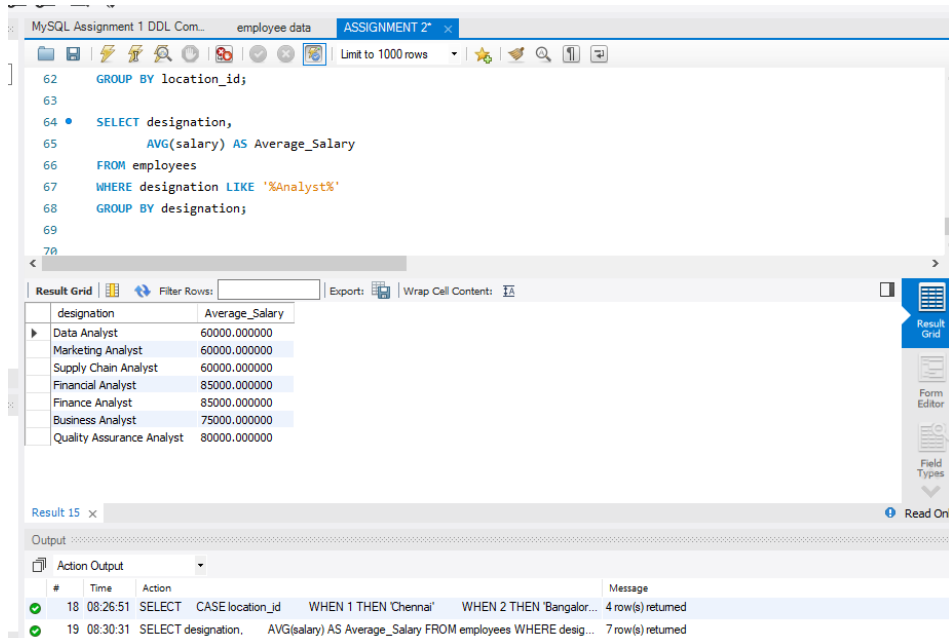
Location_Name	Maximum_Salary
Pune	100000.00
Chennai	95000.00
Bangalore	95000.00
Hyderabad	90000.00

Result 14 x

Output

#	Time	Action	Message
17	08:26:11	SELECT location_id, MAX(salary) AS Maximum_Salary FROM employees GROUP BY lo...	4 row(s) returned

b). Calculating average salary for Analyst designations.



```
62 GROUP BY location_id;
63
64 • SELECT designation,
65         AVG(salary) AS Average_Salary
66     FROM employees
67     WHERE designation LIKE '%Analyst%'
68     GROUP BY designation;
```

designation	Average_Salary
Data Analyst	60000.000000
Marketing Analyst	60000.000000
Supply Chain Analyst	60000.000000
Financial Analyst	85000.000000
Finance Analyst	85000.000000
Business Analyst	75000.000000
Quality Assurance Analyst	80000.000000

Result 15 x

Output

#	Time	Action	Message
18	08:26:51	SELECT CASE location_id WHEN 1 THEN 'Chennai' WHEN 2 THEN 'Bangalore'...	4 row(s) returned
19	08:30:31	SELECT designation, AVG(salary) AS Average_Salary FROM employees WHERE desig...	7 row(s) returned

V. HAVING

a). Finding departments with fewer than three employees

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
65 AVG(salary) AS Average_Salary
66 FROM employees
67 WHERE designation LIKE '%Analyst%'
68 GROUP BY designation;
69
70 • SELECT department_id,
71     COUNT(*) AS Employee_Count
72 FROM employees
73 GROUP BY department_id
74 HAVING COUNT(*) < 3;
75
```

The Result Grid shows the following data:

department_id	Employee_Count
11	2
13	1
7	2
9	1
1	1
5	2
6	2

The Output pane shows the execution log:

#	Time	Action	Message
19	08:30:31	SELECT designation, AVG(salary) AS Average_Salary FROM employees WHERE designation LIKE '%Analyst%' GROUP BY designation;	7 row(s) returned
20	08:32:57	SELECT department_id, COUNT(*) AS Employee_Count FROM employees GROUP BY department_id HAVING COUNT(*) < 3;	7 row(s) returned

b). Finding locations with female employees whose average age is below 30.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
76 • SELECT
77     CASE location_id
78     WHEN 1 THEN 'Chennai'
79     WHEN 2 THEN 'Bangalore'
80     WHEN 3 THEN 'Hyderabad'
81     WHEN 4 THEN 'Pune'
82     END AS Location_Name,
83     AVG(age) AS Average_Age
84 FROM employees
85 WHERE gender = 'F'
86 GROUP BY location_id
87 HAVING AVG(age) < 30;
```

The Result Grid shows the following data:

Location_Name	Average_Age
Bangalore	27.3333
Chennai	26.7500
Pune	29.2000

The Output pane shows the execution log:

#	Time	Action	Message
24	10:02:44	SELECT department_id, COUNT(*) AS Employee_Count FROM employees GROUP BY department_id HAVING COUNT(*) < 3;	7 row(s) returned
25	10:02:51	SELECT CASE location_id WHEN 1 THEN 'Chennai' WHEN 2 THEN 'Bangalore' WHEN 3 THEN 'Hyderabad' WHEN 4 THEN 'Pune' END AS Location_Name, AVG(age) AS Average_Age FROM employees WHERE gender = 'F' GROUP BY location_id HAVING AVG(age) < 30;	3 row(s) returned

JOINS

1. INNER JOIN

Using INNER JOIN to display employee and department details

The screenshot shows a MySQL Workbench window with a query editor and a result grid. The query is an INNER JOIN between the employees and departments tables. The result grid displays 30 rows of employee data, including their names, designations, and department names.

```
83 SELECT employees.employee_name,  
84 employees.designation,  
85 departments.department_name  
86 FROM employees  
87 INNER JOIN departments  
88 ON employees.department_id = departments.department_id;
```

employee_name	designation	department_name
Vhaan Singh	Data Analyst	Data Science
Ravish Singh	Network Engineer	IT
Aaradhya Iyer	Customer Support Executive	Customer Support
Kara Mahotra		Design
Anvi Chaudhary	Business Development Executive	Business Development
Dhruv Shetty	UI Developer	Design
Anushka Singh	Marketing Manager	Marketing
Divya Jha	Graphic Designer	Design
Kaani Desai	Sales Executive	Business Development
Atharv Yadav	Systems Administrator	IT
Saavi Patel	Marketing Analyst	Marketing
Myra Verma	Operations Manager	Operations
Anavi Rao	Customer Success Manager	Customer Support

Output: 30 row(s) returned

2. LEFT JOIN

Listing all departments with their employee counts using LEFT JOIN

The screenshot shows a MySQL Workbench window with a query editor and a result grid. The query is a LEFT JOIN between the departments and employees tables. The result grid displays 30 rows of department data, including their names and the names of employees assigned to them.

```
89 SELECT departments.department_name,  
90 employees.employee_name  
91 FROM departments  
92 LEFT JOIN employees  
93 ON departments.department_id = employees.department_id;
```

department_name	employee_name
Business Development	Kaani Desai
Business Development	Anvi Chaudhary
Content Creation	Anaya Kapoor
Content Creation	Ananya Paul
Customer Support	Vhaan Mohan
Customer Support	Anavi Rao
Customer Support	Aaradhya Iyer
Data Science	Vhaan Singh
Data Science	Anvi Iyengar
Data Science	Vhaan Singh
Design	Divya Jha
Design	Dhruv Shetty
Design	Kara Mahotra

Output: 30 row(s) returned

3. RIGHT JOIN

Using RIGHT JOIN to display locations and assigned employees

The screenshot shows a MySQL Workbench window with a query editor and a result grid. The query is a RIGHT JOIN between the employees and location tables. The result grid displays 30 rows of employee data, including their names and the locations they are assigned to.

```
95  
96 SELECT employees.employee_name,  
97 location.location  
98 FROM employees  
99 RIGHT JOIN location  
100 ON employees.location_id = location.location_id;
```

employee_name	location
Arjun Kumar	Bangalore
Vhaan Singh	Bangalore
Anika Paul	Bangalore
Kabir Nair	Bangalore
Vhaan Mohan	Bangalore
Myra Verma	Bangalore
Dhruv Shetty	Bangalore
Aaradhya Iyer	Bangalore
Sara Iyer	Chennai
Anaya Kapoor	Chennai
Anvi Shetty	Chennai
Anavi Rao	Chennai
Shaan Kumar	Chennai

Output: 30 row(s) returned

CONCLUSION

This assignment provided practical experience in working with MySQL and SQL queries using an employee database. Through the different exercises, I learned how to retrieve distinct values, filter and sort records, use aggregate functions, group data, and apply conditions using HAVING. The assignment also helped me understand how INNER JOIN, LEFT JOIN, and RIGHT JOIN can be used to retrieve related information from different tables. Overall, the practical exercises improved my understanding of SQL and strengthened my ability to work with relational databases using MySQL Workbench.